Cyberscope

A TAC Security Company

Audit Report

Zypto

February 2026

Network BASE

Address 0xB4D9e251B483A29b3a71e820370B9914d0774fA4

Audited by © cyberscope

Analysis

Critical Medium Minor / Informative Pass

Severity	Code	Description	Status
Critical	ST	Stops Transactions	Unresolved
Pass	OTUT	Transfers User's Tokens	Passed
Pass	ELFM	Exceeds Fees Limit	Passed
Critical	MT	Mints Tokens	Unresolved
Critical	BT	Burns Tokens	Unresolved
Pass	BC	Blacklists Addresses	Passed

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Critical	BUIBS	Balance Update Initiated Before Swap	Unresolved
Minor / Informative	CCR	Contract Centralization Risk	Unresolved
Minor / Informative	MAC	Missing Access Control	Unresolved
Minor / Informative	PLPI	Potential Liquidity Provision Inadequacy	Unresolved
Minor / Informative	PMRM	Potential Mocked Router Manipulation	Unresolved
Minor / Informative	PTRP	Potential Transfer Revert Propagation	Unresolved
Minor / Informative	PISV	Potentially Inconsistent State Variable	Unresolved
Minor / Informative	ZSS	Zero Slippage Swap	Unresolved
Minor / Informative	L04	Conformance to Solidity Naming Conventions	Unresolved
Minor / Informative	L07	Missing Events Arithmetic	Unresolved
Minor / Informative	L16	Validate Variable Setters	Unresolved

Table of Contents

Analysis **1**
Diagnostics **2**
Table of Contents **3**
Risk Classification **5**
Review **6**
Audit Updates 6
Source Files 6
Findings Breakdown **7**
ST - Stops Transactions 8
Description 8
Recommendation 8
MT - Mints Tokens 9
Description 9
Recommendation 9
BT - Burns Tokens 11
Description 11
Recommendation 12
BUIBS - Balance Update Initiated Before Swap 13
Description 13
Recommendation 13
CCR - Contract Centralization Risk 14
Description 14
Recommendation 15
MAC - Missing Access Control 16
Description 16
Recommendation 16
PLPI - Potential Liquidity Provision Inadequacy 17
Description 17
Recommendation 19
PMRM - Potential Mocked Router Manipulation 20
Description 20
Recommendation 21
PTRP - Potential Transfer Revert Propagation 22
Description 22
Recommendation 23
PISV - Potentially Inconsistent State Variable 24
Description 24
Recommendation 25
ZSS - Zero Slippage Swap 26

Description 26
Recommendation 26
L04 - Conformance to Solidity Naming Conventions 27
Description 27
Recommendation 28
L07 - Missing Events Arithmetic 29
Description 29
Recommendation 29
L16 - Validate Variable Setters 30
Description 30
Recommendation 30
Functions Analysis **31**
Inheritance Graph **33**
Flow Graph **34**
Summary **35**
Disclaimer **36**
About Cyberscope **37**

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	ZyptoToken
Compiler Version	v0.8.31+commit.fd3a2265
Optimization	200 runs
Explorer	https://basescan.org/address/0xb4d9e251b483a29b3a71e820370b9914d0774fa4
Address	0xb4d9e251b483a29b3a71e820370b9914d0774fa4
Network	BASE
Symbol	ZYPTO
Decimals	18

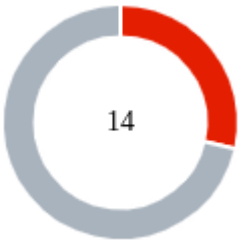
Audit Updates

Initial Audit	20 Feb 2026 https://github.com/cyberscope-io/audits/blob/main/1-zypto/v1/audit.pdf
Corrected Phase 2	23 Feb 2026

Source Files

Filename	SHA256
ZyptoToken.sol	d27dd28ff8684bf539d71adf4b6f506225b8970ad32d7fc1d79ca064abbeb65f

Findings Breakdown



Critical	4
Medium	0
Minor / Informative	10

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	4	0	0	0
Medium	0	0	0	0
Minor / Informative	10	0	0	0

ST - Stops Transactions

Criticality	Critical
Location	Zypto.sol#L236
Status	Unresolved

Description

The transactions are initially disabled for all users excluding the authorized addresses. The owner can enable the transactions for all users. Once the transactions are enabled the owner will not be able to disable them again. Furthermore, transactions can be stopped as described by the findings `PMRM` , `ZSS` , `PLPI` & `PTRP` .

```shell
function openTrade() external onlyOwner {
_isOpenForTrading = true;

}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions. Some suggestions are:

- Introduce a multi-sign wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

MT - Mints Tokens

Criticality	Critical
Location	Zypto.sol#L211
Status	Unresolved

Description

The contract owner and the `tokenPool` address have the authority to mint tokens. Both entities can achieve that by calling the `mint` function. As a result, the contract tokens will be highly inflated. Furthermore, the owner can set `tokenPool` to an arbitrary address using the `setTokenPool` method.

```shell
function mint(address account, uint256 amount) external {
  require(msg.sender == tokenPool || msg.sender == owner(),
"Not  authorized");
  _update(address(0), account, amount);

}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.
- Setting the `tokenPool` to an immutable value once defined.

BT - Burns Tokens

Criticality	Critical
Location	Zypto.sol#L216
Status	Unresolved

Description

The contract owner and the tokenPool address have the authority to burn tokens from a specific address. Both entities can achieve that by calling the `burn` function. As a result, the targeted address will lose the corresponding tokens. Furthermore, the owner can set tokenPool to an arbitrary address using the setTokenPool method.

```shell
function burn(address account, uint256 amount) external {
        require(msg.sender == tokenPool || msg.sender ==
owner(), "Not authorized");
        _update(account, address(0), amount);

    }
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.
- Setting the tokenPool to an immutable value once defined.

BUIBS - Balance Update Initiated Before Swap

Criticality	Critical
Location	Zypto.sol#L352
Status	Unresolved

Description

The contract is designed to perform a token swap after updating the respective balances. This sequence of operations can cause inconsistencies in the balances used for operations involving the router, which may result in the swap being halted. These inconsistencies disrupt the functionality of the swap, causing transactions to fail and affecting the overall liquidity operations of the contract.

```shell
function _trade(address from, address to, uint256
value) internal {
    ...
    _update(from, address(this), totalTax);
    ...
    _update(from, to, value - totalTax);
    ...
    _swapAndLiquify(contractBalance);

}
```

Recommendation

It is recommended to ensure that the swap is triggered only before the balances are updated. This will prevent inconsistencies in the balances used during router operations, ensuring the swap executes successfully and maintaining smooth token sale functionality.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	Zypto.sol#L94,206,211,216,232,236,240,244,250,255,262,269,274,279,284,449,453
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```shell
function setTokenPool(address _tokenPool) external
onlyOwner {...}
function mint(address account, uint256 amount) external
{...}
function burn(address account, uint256 amount) external
{...}
function setAutomatedMarketMakerPair(address pair, bool
value) external onlyOwner {...}
function setRouter(address router) external onlyOwner {...}
function openTrade() external onlyOwner {...}
function setBuyFee(uint256 liquidityFee, uint256
marketingFee) public onlyOwner {...}
function setSellFee(uint256 liquidityFee, uint256
marketingFee) public onlyOwner {...}
function excludeFromFees(address account, bool excluded)
public onlyOwner {...}
function setMarketingWallet(address account) external
onlyOwner {...}
function setSwapAndLiquifyEnabled(bool enabled) external
onlyOwner {...}
function setSwapSlippage(uint256 bps) external onlyOwner
```

```
{...}
function setSwapTokensAtAmount(uint256 amount) external
onlyOwner {...}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

MAC - Missing Access Control

Criticality	Minor / Informative
Location	ZyptoToken.sol#L284
Status	Unresolved

Description

The `swapAndLiquifyNow` function has no access control, allowing anyone to trigger the swap of the contract's entire accumulated fee token balance at any time. This enables third parties to trigger unauthorized swaps potentially exposing the contact to scenarios of MEV manipulation.

```shell
function swapAndLiquifyNow() external {
    uint256 contractBalance =
balanceOf(address(this));
    require(contractBalance > 0, "Nothing to
swap");
    _swapAndLiquify(contractBalance);

}
```

Recommendation

The team is advised to implement proper access control ensuring only administrative authorities can execute this function.

PLPI - Potential Liquidity Provision Inadequacy

Criticality	Minor / Informative
Location	Zypto.sol#L416,433
Status	Unresolved

Description

The contract operates under the assumption that liquidity is consistently provided to the pair between the contract's token and the native currency. However, there is a possibility that liquidity is provided to a different pair. This inadequacy in liquidity provision in the main pair could expose the contract to risks. Specifically, during eligible transactions, where the contract attempts to swap tokens with the main pair, a failure may occur if liquidity has been added to a pair other than the primary one. Consequently, transactions triggering the swap functionality will result in a revert.

Shell

```
function _swapTokensForETH(uint256 tokenAmount, address to)
private {
        address[] memory path = new address[](2);
        path[0] = address(this);
        path[1] = _WETH;

        uint256 minOut =
_uniswapV2Router.getAmountsOut(tokenAmount, path)[1]
            .calcPortionFromBasisPoints(10000 -
_swapSlippageBps);


_uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransf
erTokens(
            tokenAmount,
            minOut,
            path,
            to,
            block.timestamp
        );
}

function _addLiquidity(uint256 tokenAmount, uint256
ethAmount) private {
        uint256 minToken =
tokenAmount.calcPortionFromBasisPoints(10000 -
_swapSlippageBps);
        uint256 minEth =
ethAmount.calcPortionFromBasisPoints(10000 -
_swapSlippageBps);

        _uniswapV2Router.addLiquidityETH{value: ethAmount}(
            address(this),
            tokenAmount,
            minToken,
            minEth,
            owner(),
            block.timestamp);}
```

Recommendation

The team is advised to implement a runtime mechanism to check if the pair has adequate liquidity provisions. This feature allows the contract to omit token swaps if the pair does not have adequate liquidity provisions, significantly minimizing the risk of potential failures.

Furthermore, the team could ensure the contract has the capability to switch its active pair in case liquidity is added to another pair.

Additionally, the contract could be designed to tolerate potential reverts from the swap functionality, especially when it is a part of the main transfer flow. This can be achieved by executing the contract's token swaps in a non-reversible manner, thereby ensuring a more resilient and predictable operation.

PMRM - Potential Mocked Router Manipulation

Criticality	Minor / Informative
Location	Zypto.sol#L223
Status	Unresolved

Description

The contract includes a method that allows the owner to modify the router address and create a new pair. While this feature provides flexibility, it introduces a security threat. The owner could set the router address to any contract that implements the router's interface, potentially containing malicious code. In the event of a transaction triggering the swap functionality with such a malicious contract as the router, the transaction may be manipulated.

```shell
function _setupRouter(address router) private {
    _uniswapV2Router = IUniswapV2Router02(router);
    _WETH = _uniswapV2Router.WETH();
    _uniswapV2Pair =
IUniswapV2Factory(_uniswapV2Router.factory()).createPair(ad
dress(this), _WETH);
    _isAutomatedMarketMakerPair[_uniswapV2Pair] = true;
    _approve(address(this), address(_uniswapV2Router),
type(uint256).max);

}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

PTRP - Potential Transfer Revert Propagation

Criticality	Minor / Informative
Location	Zypto.sol#L274,424
Status	Unresolved

Description

The contract sends funds to a `marketingWallet` as part of the transfer flow. This address can either be a wallet address or a contract. If the address belongs to a contract then it may revert from incoming payment. As a result, the error will propagate to the token's contract and revert the transfer.

```shell
_uniswapV2Router.swapExactTokensForETHSupportingFee
OnTransferTokens(
            tokenAmount,
            minOut,
            path,
            to,
            block.timestamp
        );

    }
```

```shell
function setMarketingWallet(address account)
external onlyOwner {
        _marketingWallet = payable(account);
        _isExcludedFromFees[account] = true;

    }
```

Recommendation

The contract should tolerate the potential revert from the underlying contracts when the interaction is part of the main transfer flow. This could be achieved by not allowing set contract addresses or by sending the funds in a non-revertable way, including try-catch schemes.

PISV - Potentially Inconsistent State Variable

Criticality	Minor / Informative
Location	Zypto.sol#L449
Status	Unresolved

Description

The contract provides a `rescueERC20` function that allows the owner to withdraw the entire balance of any ERC20 token held by the contract. However, if the rescued token is the contract's own token (address(this)), the `_pendingMarketingValue` state variable is never reset to zero. This creates an accounting inconsistency where the variable continues to track a marketing allocation that no longer exists in the contract's balance.

When fees subsequently accumulate and `_swapAndLiquify` is triggered, the stale `_pendingMarketingValue` will cause a disproportionate amount of tokens to be allocated toward the marketing swap, effectively shortchanging the liquidity provision.

```shell
function rescueERC20(IERC20 _token) external
onlyOwner {
    _token.safeTransfer(msg.sender,
_token.balanceOf(address(this)));

}
```

Recommendation

The contract should reset `_pendingMarketingValue` to zero when the rescued token is the contract's own token. Alternatively, the contract could restrict rescueERC20 from withdrawing its own token entirely, limiting the function to recovering only third-party tokens accidentally sent to the contract.

ZSS - Zero Slippage Swap

Criticality	Minor / Informative
Location	ZyptoToken.sol#L250
Status	Unresolved

Description

The contract allows the owner to set the swap slippage tolerance to zero via the `setSwapSlippage` function. The validation only enforces an upper bound `(bps <= 1000)` but does not enforce a minimum value. When `_swapSlippageBps` is set to `0`, the `minOut` calculation in `_swapTokensForETH` becomes 100% of the quoted amount from `getAmountsOut`. This may lead the swap to fail, effectively stopping transactions that trigger the auto-swap mechanism.

```shell
function setSwapSlippage(uint256 bps) external
onlyOwner {
        require(bps <= 1000, "Slippage too high");
        _swapSlippageBps = bps;

    }
```

Recommendation

The team is advised to ensure the contract applies a minimum slippage tolerance at all times. This will ensure consistency and prevent the swap mechanism from failing during eligible transfers.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	ZyptoToken.sol#L65,118,150,151,152,155,156,157,158,159,160,161,162,163,165,166,169,170,171,210,453
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```shell
uint256 _basisPoints
uint256 _amount
function WETH() external pure returns (address);
IUniswapV2Router02 public _uniswapV2Router
address public _WETH
uint256 public _swapSlippageBps = 200;
address public _uniswapV2Pair
uint256 public _buyFeeLiquidity
uint256 public _buyFeeMarketing
uint256 public _buyFeeTotal
uint256 public _sellFeeLiquidity
uint256 public _sellFeeMarketing
uint256 public _sellFeeTotal
uint256 public _pendingMarketingValue
address payable public _marketingWallet


...
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions.

L07 - Missing Events Arithmetic

Criticality	Minor / Informative
Location	ZyptoToken.sol#L245,256,260,267
Status	Unresolved

Description

Events are a way to record and log information about changes or actions that occur within a contract. They are often used to notify external parties or clients about events that have occurred within the contract, such as the transfer of tokens or the completion of a task.

It's important to carefully design and implement the events in a contract, and to ensure that all required events are included. It's also a good idea to test the contract to ensure that all events are being properly triggered and logged.

```shell
_swapTokensAtAmount = amount
_swapSlippageBps = bps
_buyFeeLiquidity = liquidityFee

_sellFeeLiquidity = liquidityFee
```

Recommendation

By including all required events in the contract and thoroughly testing the contract's functionality, the contract ensures that it performs as intended and does not have any missing events that could cause issues with its arithmetic.

L16 - Validate Variable Setters

Criticality	Minor / Informative
Location	ZyptoToken.sol#L188,211,279
Status	Unresolved

Description

The contract performs operations on variables that have been configured on user-supplied input. These variables are missing a proper check for the case where a value is zero. This can lead to problems when the contract is executed, as certain actions may not be properly handled when the value is zero.

```shell
_marketingWallet = payable(marketing)
tokenPool = _tokenPool

_marketingWallet = payable(account)
```

Recommendation

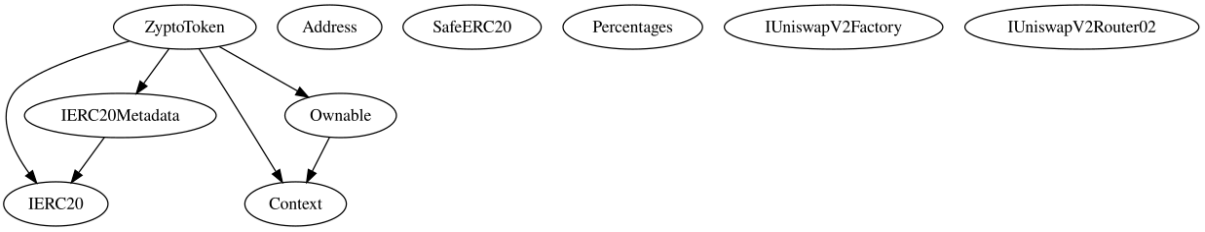
By adding the proper check, the contract will not allow the variables to be configured with zero value. This will ensure that the contract can handle all possible input values and avoid unexpected behavior or errors. Hence, it can help to prevent the contract from being exploited or operating unexpectedly.

Functions Analysis

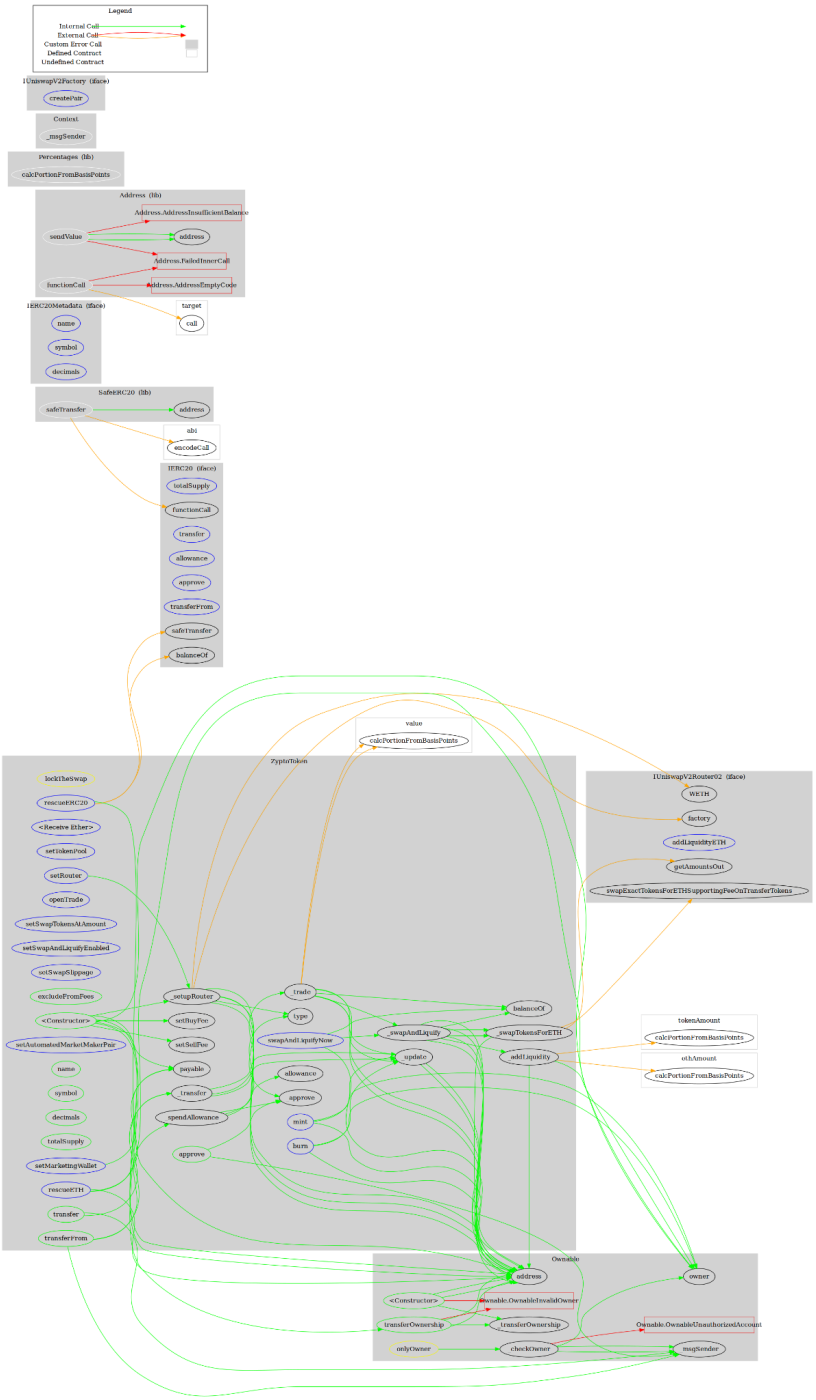
Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
Percentages	Library			
	calcPortionFromBasisPoints	Internal		
ZyptoToken	Implementation	Context, IERC20, IERC20Metadata, Ownable		
		Public	✓	Ownable
		External	Payable	-
	setTokenPool	External	✓	onlyOwner
	mint	External	✓	-
	burn	External	✓	-
	_setupRouter	Private	✓	
	setRouter	External	✓	onlyOwner
	openTrade	External	✓	onlyOwner
	setSwapTokensAtAmount	External	✓	onlyOwner
	setSwapAndLiquifyEnabled	External	✓	onlyOwner
	setSwapSlippage	External	✓	onlyOwner
	setBuyFee	Public	✓	onlyOwner

	setSellFee	Public	✓	onlyOwner
	excludeFromFees	Public	✓	onlyOwner
	setMarketingWallet	External	✓	onlyOwner
	setAutomatedMarketMakerPair	External	✓	onlyOwner
	swapAndLiquifyNow	External	✓	-
	name	Public		-
	symbol	Public		-
	decimals	Public		-
	totalSupply	Public		-
	balanceOf	Public		-
	transfer	Public	✓	-
	allowance	Public		-
	approve	Public	✓	-
	transferFrom	Public	✓	-
	_approve	Internal	✓	
	_spendAllowance	Internal	✓	
	_transfer	Internal	✓	
	_trade	Internal	✓	
	_update	Internal	✓	
	_swapAndLiquify	Private	✓	lockTheSwap
	_swapTokensForETH	Private	✓	
	_addLiquidity	Private	✓	
	rescueERC20	External	✓	onlyOwner
	rescueETH	External	✓	onlyOwner

Inheritance Graph



Flow Graph


Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
IUniswapV2Factory (iface)
createPair
Context
_msgSender
Percentages (lib)
calcPortionFromBasisPoints
Address (lib)
Address.AddressInsufficientBalance
sendValue
address
Address.FailedInnerCall
functionCall
Address.AddressEmptyCode
IERC20Metadata (iface)
name
symbol
decimals
target
call
SafeERC20 (lib)
safeTransfer
address
abi
encodeCall
IERC20 (iface)
totalSupply
functionCall
transfer
allowance
approve
transferFrom
safeTransfer
balanceOf
value
calcPortionFromBasisPoints
ZyptoToken
lockTheSwap
rescueERC20
<Receive Ether>
setTokenPool
setRouter
openTrade
setSwapTokensAtAmount
setSwapAndLiquifyEnabled
setSwapSlippage
excludeFromFees
<Constructor>
setAutomatedMarketMakerPair
name
symbol
decimals
totalSupply
setMarketingWallet
rescueETH
transfer
transferFrom
_setupRouter
setBuyFee
setSellFee
payable
transfer
spendAllowance
approve
trade
type
swapAndLiquifyNow
allowance
approve
mint
burn
balanceOf
_swapAndLiquify
_swapTokensForETH
_update
addLiquidity
IUniswapV2Router02 (iface)
WETH
factory
addLiquidityETH
getAmountsOut
swapExactTokensForETHSupportingFeeOnTransferTokens
tokenAmount
calcPortionFromBasisPoints
ethAmount
calcPortionFromBasisPoints
Ownable
address
<Constructor>
Ownable.OwnableInvalidOwner
transferOwnership
transferOwnership
onlyOwner
checkOwner
Ownable.OwnableUnauthorizedAccount
owner
msgSender

Summary

Zypto contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the owner like stop transactions, mint tokens and burn tokens from any address. If the contract owner abuses the mint functionality, then the contract will be highly inflated. If the contract owner abuses the burn functionality, then the users could lose their tokens. A multi-wallet signing pattern will provide security against potential hacks. Temporarily locking the contract or renouncing ownership will eliminate all the contract threats. There is also a limit of max 4% fees.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io